

Homework 5: [YOUR NAME HERE]

Introduction to Time Series, Fall 2024

2001-11-22

The total number of points possible for this homework is 48. The number of points for each question is written below, and questions marked as “bonus” are optional (points awarded for bonus problems can be used to earn back points that you may have lost on other parts of this homework but will not put you above full credit). Submit the **knitted pdf file** from this Quarto document to Gradescope.

If you collaborated with anybody for this homework, put their names here:

Notes about this homework

This homework is an exercise in Covid-19 forecasting. You will build some basic forecasters of Covid-19 deaths at the national (U.S.) level, based on frameworks you’ve learned in the last few lectures: ARIMA and ETS. You should know that, in true (prospective) Covid-19 forecasting, the situation is much harder than the one you are facing in this homework. This is because of **data revisions**: the forecasters in true (prospective) Covid-19 forecasting did not have access to the same data in real-time that you have access to now, in retrospect. Instead, they had access to preliminary data that was subject to revisions, sometimes very large and irregular ones, making forecasting much harder. See, e.g., [McDonald et al. \(2021\)](#), for a discussion of the impact of revisions on forecasting.

Also, in most operational forecasting enterprises, epidemic/pandemic forecasting included, we would typically be trying to leverage exogenous signals and sources of information to guide to our forecasts (beyond statistical models like ARIMA or ETS which only rely on historical information about the target of interest). Here we will pursue this only in limited fashion, at the end of this homework.

Covid-19 cases and deaths

First, download the data files `cases_deaths.csv` and `covidhub_fc.csv` from the course GitHub repo. The former contains weekly reported Covid-19 cases and death counts in the U.S., from February 29, 2020 to March 4, 2023. The latter contains 1-4 week ahead forecasts of weekly reported Covid-19 death counts, from June 13, 2020 to March 4, 2023. These forecasts were generated by the CovidHub ensemble model, which is an ensemble of all qualifying forecast submissions to the U.S. Covid-19 Forecast Hub, and was the basis of official CDC communications during the pandemic. See, e.g., [Ray et al. \(2022\)](#) for discussion of the ensemble model.

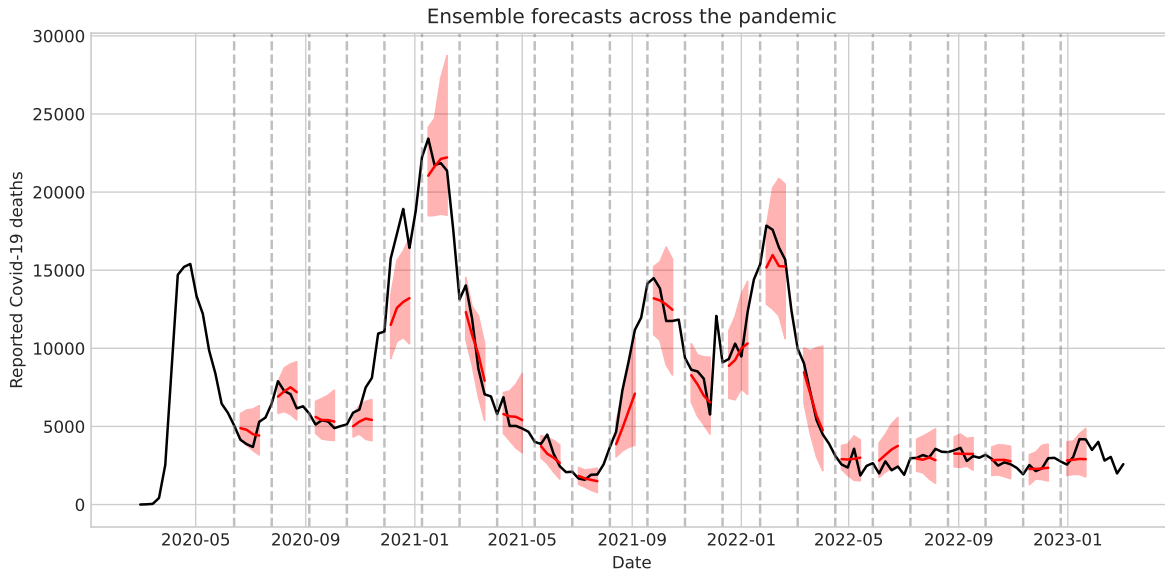
The code below loads in these data frames (which you can run once the downloaded files are in your working directory) and plots death curves along a set of 1-4 week ahead forecasts, across the pandemic.

```
cases_deaths = pd.read_csv("cases_deaths.csv")
covidhub_fc = pd.read_csv("covidhub_fc.csv")

cases_deaths["date"] = pd.to_datetime(cases_deaths["date"])
covidhub_fc["target_date"] = pd.to_datetime(covidhub_fc["target_date"])

# Calculate forecast date: target_date - h * 7 days
covidhub_fc["forecast_date"] = covidhub_fc["target_date"] - pd.to_timedelta(covidhub_fc["h"])

plot_forecast_trajectories(
    cases_deaths,
    covidhub_fc,
    "Ensemble forecasts across the pandemic",
    mean_col="forecast_0.5",
    lower_col="forecast_0.1",
    upper_col="forecast_0.9"
)
```



As you can see from the plot, the ensemble model produces both point forecasts and prediction intervals. In fact, the prediction intervals are derived from predicted quantiles. The point forecast is stored in the `forecast_0.5` column, whereas the 80% prediction interval is formed by taking the difference of the `forecast_0.9` and `forecast_0.1` columns (and similarly for other levels). Lastly, the `h` column indicates horizon of the forecast (in weeks), and the `target_date` column denotes the target date (aligned by the end of the week), whose death count the ensemble model is predicting. The forecast date can hence be derived from `target_date - h * 7` (which, since `target_date` is stored as a `Date` variable, will be understood correctly to mean subtract off `h * 7` days from the given date).

Evaluating the ensemble model's MAE

1. (4 pts) Before building any forecasters of our own, we'll evaluate the ensemble model's mean absolute error (MAE). We won't be able to beat this with our models, but it'll give us a sense of how gold-standard forecasts perform, for this problem. First, you need join the death data to the forecast data. Using `left_join()`, join `covidhub_fc` and `cases_deaths` by date: specifically, the `target_date` variable in the first data set should be matched to the `date` variable in the second. As a check, you should see for that the 4 week ahead prediction for the target date July 4, 2020, the point forecast is 5169.654 and the death count is 3684. After joining the data sets, compute the MAE of the point forecasts per horizon, report the results.

```
# Join covidhub_fc and cases_deaths by target_date == date
merged = pd.merge(covidhub_fc, cases_deaths, left_on="target_date", right_on="date", how="left")
```

```

# Check for target date July 4, 2020, horizon h = 4
check_row = merged[(merged["target_date"] == "2020-07-04") & (merged["h"] == 4)]
if not check_row.empty:
    fc_val = check_row["forecast_0.5"].values[0]
    death_val = check_row["deaths"].values[0]
    print(f"Check July 4, 2020 h=4: Forecast = {fc_val:.3f}, Deaths = {death_val}")

# Compute MAE per horizon
print("Ensemble MAE per horizon:")
ensemble_maes = []
for h in [1, 2, 3, 4]:
    sub = merged[merged["h"] == h].dropna(subset=["deaths"])
    mae = np.mean(np.abs(sub["forecast_0.5"] - sub["deaths"]))
    ensemble_maes.append(mae)
    print(f"  h {h}: {mae:.2f}")

```

Check July 4, 2020 h=4: Forecast = 5169.654, Deaths = 3684

Ensemble MAE per horizon:

```

h 1: 822.78
h 2: 994.51
h 3: 1196.60
h 4: 1424.33

```

(Type your discussion here)

2. (2 pts) Using the same joined data set from Q1, compute the coverage of 80% prediction intervals, per horizon, and report the results.

```

# Compute 80% coverage per horizon
ensemble_coverages = []
for h in [1, 2, 3, 4]:
    sub = merged[merged["h"] == h].dropna(subset=["deaths", "forecast_0.1", "forecast_0.9"])
    covered = (sub["deaths"] >= sub["forecast_0.1"]) & (sub["deaths"] <= sub["forecast_0.9"])
    coverage = np.mean(covered)
    ensemble_coverages.append(coverage)
    print(f"  h {h}: {coverage:.2%}")

```

```

h 1: 85.31%
h 2: 83.80%
h 3: 82.98%
h 4: 84.29%

```

(Type your discussion here)

Evaluating the baseline model's MAE

3. (5 pts) As one more thing to do before building our own models, we'll develop a simple baseline model and compute its MAE. Make a copy of the joined data from Q1, and overwrite the point forecasts with the true value of deaths from `h` weeks ago. In other words, this baseline simply predicts (at all horizons) that we will see the same number of new reported Covid-19 deaths as what we observed in the last week. (Its forecast trajectories are thus flat lines, extrapolating forward from the latest observation.) After you've done this, compute the MAE of the baseline forecasts, per horizon, and report the results.

```
# Map date to deaths for easy baseline lookup
date_to_deaths = dict(zip(cases_deaths["date"], cases_deaths["deaths"]))

merged["forecast_date"] = merged["target_date"] - pd.to_timedelta(merged["h"] * 7, unit="D")
merged["baseline_fc"] = merged["forecast_date"].map(date_to_deaths)

baseline_maes = []
for h in [1, 2, 3, 4]:
    sub = merged[merged["h"] == h].dropna(subset=["deaths", "baseline_fc"])
    mae = np.mean(np.abs(sub["baseline_fc"] - sub["deaths"]))
    baseline_maes.append(mae)
    print(f"  h {h}: {mae:.2f}")
```

```
h 1: 967.66
h 2: 1528.44
h 3: 2081.77
h 4: 2635.04
```

(Type your discussion here)

4. (Bonus) Develop a method to form an 80% prediction interval around the baseline model's point forecasts. This should still be an ex-ante prediction in the sense that it should not be using data after the forecast date. Once you've done this, compute the coverage of these prediction intervals, per horizon, and report the results.

```

# Ex-ante baseline 80% prediction intervals based on historical forecast errors
baseline_coverages = []
coverages = {1: [], 2: [], 3: [], 4: []}

# Sort cases_deaths by date to ensure proper chronology
cases_deaths_sorted = cases_deaths.sort_values("date").reset_index(drop=True)

for idx, row in merged.iterrows():
    h = row["h"]
    fc_date = row["forecast_date"]
    target_val = row["deaths"]
    if pd.isna(target_val) or pd.isna(row["baseline_fc"]):
        continue

    # Slice historical deaths up to the forecast date
    past_data = cases_deaths_sorted[cases_deaths_sorted["date"] <= fc_date]
    past_deaths = past_data["deaths"].values

    # Calculate historical errors at horizon h
    errors = []
    for s in range(h, len(past_deaths)):
        errors.append(past_deaths[s] - past_deaths[s-h])

    if len(errors) >= 5:
        q10 = np.percentile(errors, 10)
        q90 = np.percentile(errors, 90)
        ci_lower = row["baseline_fc"] + q10
        ci_upper = row["baseline_fc"] + q90
        coverages[h].append(ci_lower <= target_val <= ci_upper)

for h in [1, 2, 3, 4]:
    coverage = np.mean(coverages[h]) if coverages[h] else np.nan
    baseline_coverages.append(coverage)
    print(f"  h {h}: {coverage:.2%}")

```

```

h 1: 83.92%
h 2: 85.21%
h 3: 87.23%
h 4: 86.43%

```

```

# Plot baseline forecasts and 80% prediction intervals across the pandemic
fc_dates = pd.date_range(start="2020-06-13", end="2023-01-01", freq="42D")

baseline_records = []
for fc_date in fc_dates:
    past_df = cases_deaths_sorted[cases_deaths_sorted["date"] <= fc_date]
    if past_df.empty:
        continue
    observed_val = past_df.iloc[-1]["deaths"]
    past_deaths = past_df["deaths"].values

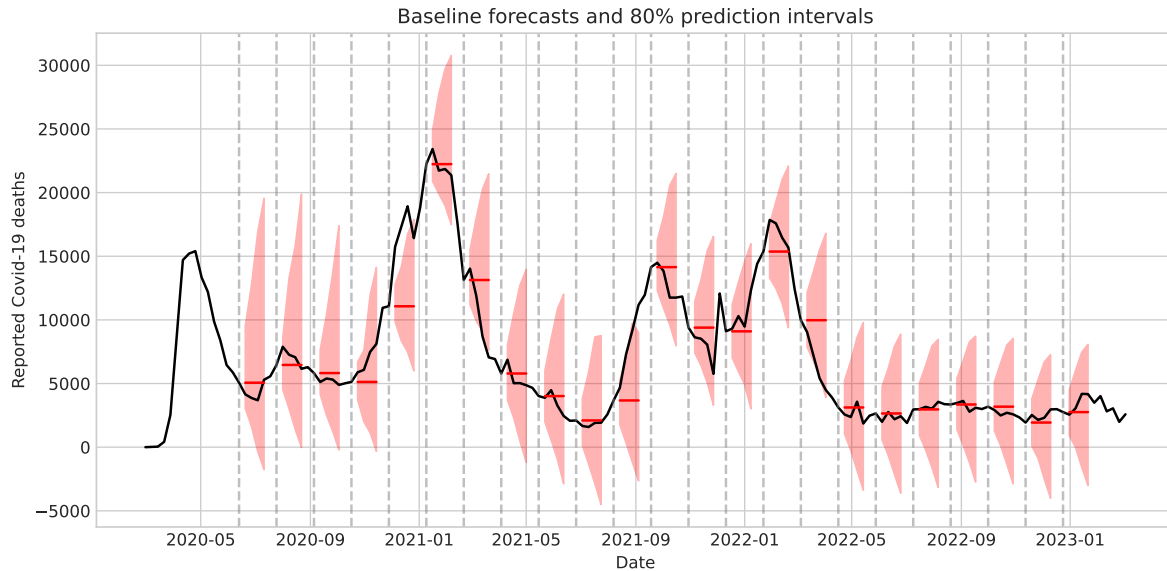
    for h in [1, 2, 3, 4]:
        errors = [past_deaths[s] - past_deaths[s-h] for s in range(h, len(past_deaths))]
        if len(errors) >= 5:
            q10 = np.percentile(errors, 10)
            q90 = np.percentile(errors, 90)
            ci_lower = observed_val + q10
            ci_upper = observed_val + q90
        else:
            ci_lower = observed_val
            ci_upper = observed_val

        baseline_records.append({
            "forecast_date": fc_date,
            "target_date": fc_date + pd.to_timedelta(h * 7, unit="D"),
            "mean": observed_val,
            "lower": ci_lower,
            "upper": ci_upper
        })

baseline_fc_df = pd.DataFrame(baseline_records)

plot_forecast_trajectories(
    cases_deaths_sorted,
    baseline_fc_df,
    "Baseline forecasts and 80% prediction intervals"
)

```



(Type your discussion here)

ARIMA models

5. (2 pts) Now we'll start building our own models, starting with ARIMA. Because the data is highly nonstationary (recall the plot above which showed death counts along with the ensemble forecasts), we'll consider differencing. Plot the first and second differences of the death data, and comment on what you find.

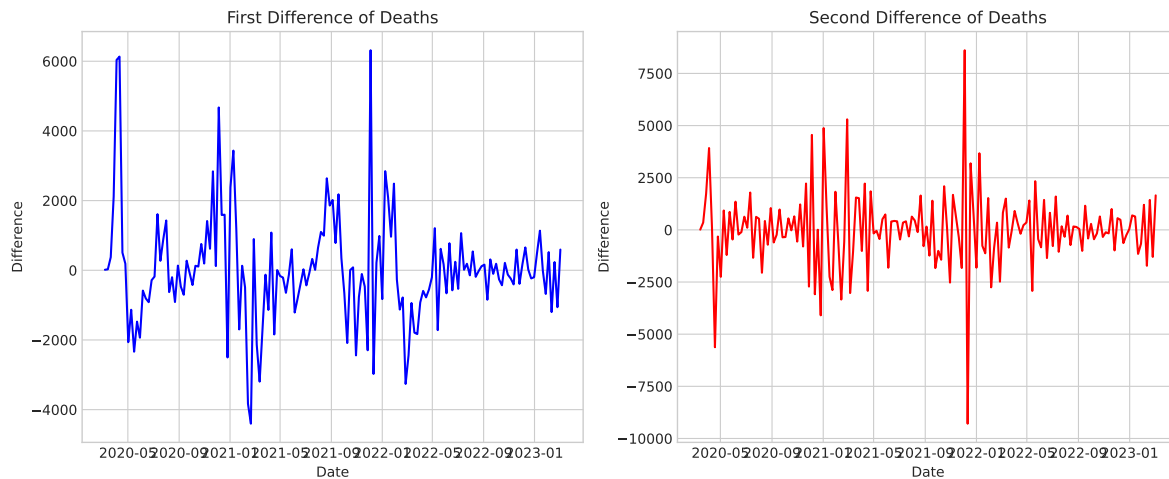
```
# Plot differenced series
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(cases_deaths_sorted["date"], cases_deaths_sorted["deaths"].diff(), color="blue")
plt.title("First Difference of Deaths")
plt.xlabel("Date")
plt.ylabel("Difference")

plt.subplot(1, 2, 2)
plt.plot(cases_deaths_sorted["date"], cases_deaths_sorted["deaths"].diff().diff(), color="red")
plt.title("Second Difference of Deaths")
plt.xlabel("Date")
plt.ylabel("Difference")
```



```
plt.tight_layout()
plt.show()
```



(Type your discussion here) The first difference removes the non-stationary trend, although clear periods of high volatility remain during the peaks of the pandemic waves. The second difference is also highly stationary and fluctuates closely around zero. This suggests that second-differencing may be better.

6. (5 pts) Fit two models: $ARIMA(1,1,0)$ and $ARIMA(2,1,0)$, using `ARIMA()` from the `fable` package. Each model should be fit only using the data up through June 6, 2020. (This is also therefore the forecast date.) These models should be fit *with* the inclusion of a constant but *without* any seasonality terms (specify the formula carefully to ensure this). After fitting these models, use these to make 1-4 week ahead forecasts and plot them (along with the true death counts through June 6, 2020) with `autoplot()`. Hint: before calling `fable::model()` you'll have to convert `cases_deaths` to a `tsibble` object, with `index = date`.

```
# Train data up to June 6, 2020
train_data = cases_deaths_sorted[cases_deaths_sorted["date"] <= "2020-06-06"]
y_train = train_data["deaths"].values

# Fit models with trend='t' (which represents the drift/constant of the differenced series)
fit_110 = ARIMA(y_train, order=(1, 1, 0), trend='t').fit()
fc_110 = fit_110.forecast(steps=4)

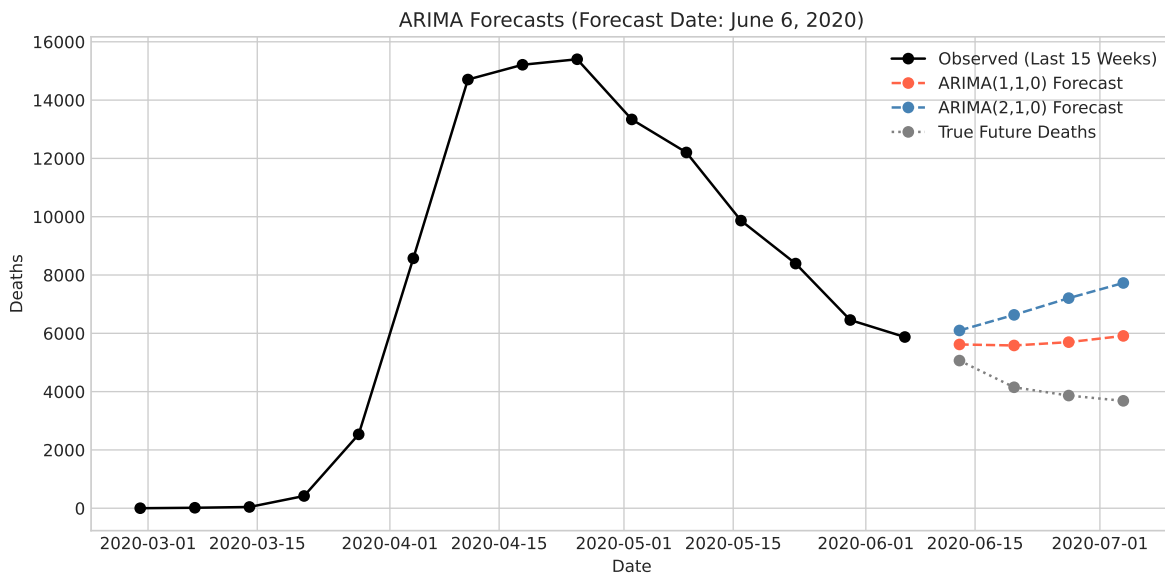
fit_210 = ARIMA(y_train, order=(2, 1, 0), trend='t').fit()
fc_210 = fit_210.forecast(steps=4)
```

```
# Create index dates for the forecasts
fc_index = pd.date_range(start="2020-06-13", periods=4, freq="7D")

# True future values for visual reference
future_true = cases_deaths_sorted[(cases_deaths_sorted["date"] > "2020-06-06") & (cases_deaths_sorted["date"] < "2020-07-01")]

plt.figure(figsize=(10, 5))
plt.plot(train_data["date"].iloc[-15:], y_train[-15:], color="black", marker="o", label="Observed (Last 15 Weeks)")
plt.plot(fc_index, fc_110, color="tomato", marker="o", linestyle="--", label="ARIMA(1,1,0) Forecast")
plt.plot(fc_index, fc_210, color="steelblue", marker="o", linestyle="--", label="ARIMA(2,1,0) Forecast")
plt.plot(future_true["date"], future_true["deaths"], color="grey", marker="o", linestyle=":", label="True Future Deaths")

plt.xlabel("Date")
plt.ylabel("Deaths")
plt.title("ARIMA Forecasts (Forecast Date: June 6, 2020)")
plt.legend()
plt.tight_layout()
plt.show()
```



7. (4 pts) For the two models from Q6, check that their 1 week ahead point forecasts match the formulas you know underlie them. To do so, extract the coefficients from the fitted models, form the 1 week ahead predictions manually, and demonstrate that these match the given forecasts.

```

# Last observed values
y_t = y_train[-1]
y_t_1 = y_train[-2]
y_t_2 = y_train[-3]

# ARIMA(1,1,0) manual prediction
mu_1 = fit_110.params[0] # trend coefficient (constant of differenced series)
phi_1 = fit_110.params[1] # ar.L1 coefficient
pred_diff_110 = mu_1 * (1 - phi_1) + phi_1 * (y_t - y_t_1)
manual_fc_110 = y_t + pred_diff_110

# ARIMA(2,1,0) manual prediction
mu_2 = fit_210.params[0] # trend coefficient
phi1_2 = fit_210.params[1] # ar.L1 coefficient
phi2_2 = fit_210.params[2] # ar.L2 coefficient
pred_diff_210 = mu_2 * (1 - phi1_2 - phi2_2) + phi1_2 * (y_t - y_t_1) + phi2_2 * (y_t_1 - y_t_2)
manual_fc_210 = y_t + pred_diff_210

print(f"ARIMA(1,1,0) 1-week ahead forecast: Statsmodels = {fc_110[0]:.6f}, Manual = {manual_fc_110:.6f}")
print(f"ARIMA(2,1,0) 1-week ahead forecast: Statsmodels = {fc_210[0]:.6f}, Manual = {manual_fc_210:.6f}")

```

ARIMA(1,1,0) 1-week ahead forecast: Statsmodels = 5617.868044, Manual = 5617.868044

ARIMA(2,1,0) 1-week ahead forecast: Statsmodels = 6097.244124, Manual = 6097.244124

(Type your discussion here)

8. (6 pts) Implement time series cross-validation (CV) for these ARIMA models, ARIMA(1,1,0) and ARIMA(2,1,0), each with constants (and no seasonality). The simplest way to do this will be write a loop.

The code below provides a scaffold that you should build on, where we run time series CV over June 6, 2020 to February 4, 2023. Make sure to explicitly define the horizon `h` in forecasts stored in the `fable_fc` list. The end result of this question will be a DataFrame, called `fable_fc_df`, which contains all of the 1-4 week ahead forecasts made in time series CV by your two ARIMA models. We will continue to append forecasts to this DataFrame in subsequent questions, as we build more models.

```

t0 = pd.to_datetime("2020-06-06")
t1 = pd.to_datetime("2023-02-04")
fc_dates = cases_deaths_sorted[(cases_deaths_sorted["date"] >= t0) & (cases_deaths_sorted["date"] < t1)]
fc_dates = pd.to_datetime(fc_dates)

```

```

fable_fc = []

for i in range(len(fc_dates)):
    fc_date = fc_dates[i]

    # Construct data set up through the forecast date
    train_idx = cases_deaths_sorted[cases_deaths_sorted["date"] == fc_date].index[0]
    train_df = cases_deaths_sorted.iloc[:train_idx + 1]
    y_tr = train_df["deaths"].values

    # Slice the actual values for the next 4 weeks
    future_df = cases_deaths_sorted.iloc[train_idx + 1 : train_idx + 5]
    actual_deaths = future_df["deaths"].values

    # 1. Fit ARIMA(1,1,0)
    try:
        fit_110 = ARIMA(y_tr, order=(1, 1, 0), trend='t').fit()
        fc_res_110 = fit_110.get_forecast(steps=len(future_df))
        mean_110 = fc_res_110.predicted_mean
        ci_110 = fc_res_110.summary_frame(alpha=0.20) # 80% CI
        for h_idx in range(len(future_df)):
            fable_fc.append({
                "model": "ARIMA(1,1,0)", "forecast_date": fc_date,
                "target_date": future_df.iloc[h_idx]["date"], "h": h_idx + 1,
                "mean": mean_110[h_idx], "lower": ci_110["mean_ci_lower"].values[h_idx],
                "upper": ci_110["mean_ci_upper"].values[h_idx], "actual": actual_deaths[h_idx]
            })
    except Exception as e:
        pass

    # 2. Fit ARIMA(2,1,0)
    try:
        fit_210 = ARIMA(y_tr, order=(2, 1, 0), trend='t').fit()
        fc_res_210 = fit_210.get_forecast(steps=len(future_df))
        mean_210 = fc_res_210.predicted_mean
        ci_210 = fc_res_210.summary_frame(alpha=0.20)
        for h_idx in range(len(future_df)):
            fable_fc.append({
                "model": "ARIMA(2,1,0)", "forecast_date": fc_date,
                "target_date": future_df.iloc[h_idx]["date"], "h": h_idx + 1,
                "mean": mean_210[h_idx], "lower": ci_210["mean_ci_lower"].values[h_idx],
                "upper": ci_210["mean_ci_upper"].values[h_idx], "actual": actual_deaths[h_idx]
            })
    except Exception as e:
        pass

```

```

    })
except Exception as e:
    pass

fable_fc_df = pd.DataFrame(fable_fc)

```

9. (4 pts) Compute the MAE of the time series CV forecasts made by each ARIMA model, per horizon. Compare this to the previously-computed MAEs of the ensemble and baseline models by displaying their MAE curves, as a function of horizon, all on the same plot. According to MAE, your ARIMA models should be better than the baseline, but notably worse than the ensemble.

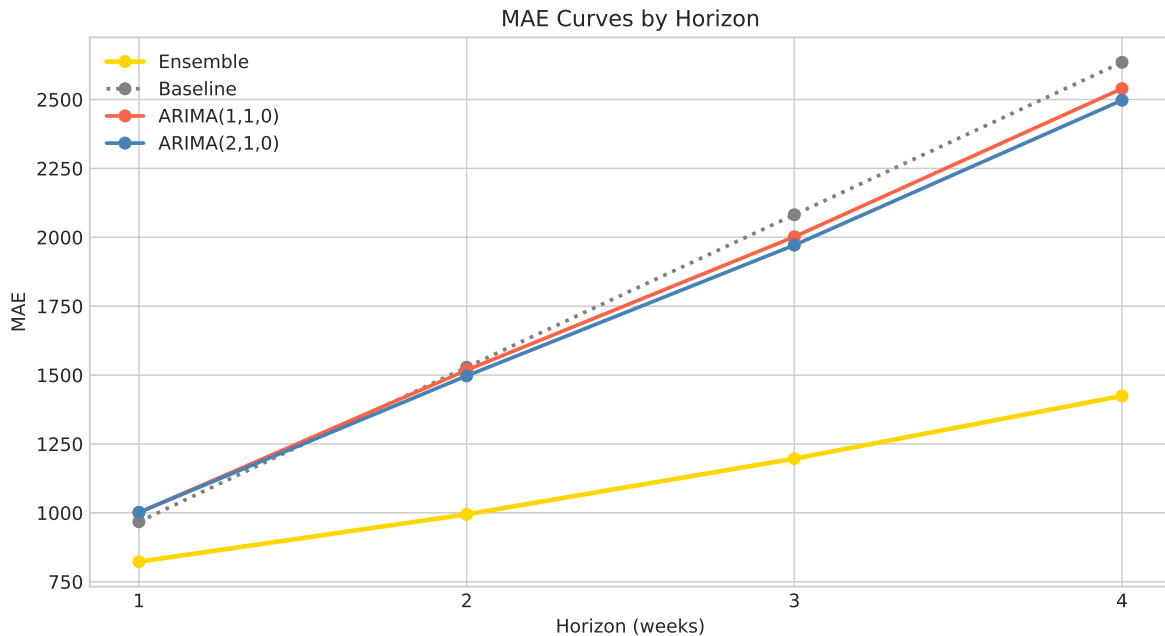
```

# Compute MAE per model per horizon
arima_maes = {"ARIMA(1,1,0)": [], "ARIMA(2,1,0)": []}
for model_name in ["ARIMA(1,1,0)", "ARIMA(2,1,0)"]:
    for h in [1, 2, 3, 4]:
        sub = fable_fc_df[(fable_fc_df["model"] == model_name) & (fable_fc_df["h"] == h)]
        mae = np.mean(np.abs(sub["mean"] - sub["actual"]))
        arima_maes[model_name].append(mae)

horizons = [1, 2, 3, 4]
plt.figure(figsize=(9, 5))
plt.plot(horizons, ensemble_maes, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_maes, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_maes["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_maes["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("MAE")
plt.title("MAE Curves by Horizon")
plt.legend()
plt.tight_layout()
plt.show()

```



(Type your discussion here)

10. (Bonus) Do the same as in Q9, but substituting MAE with coverage of the 80% prediction intervals.

```

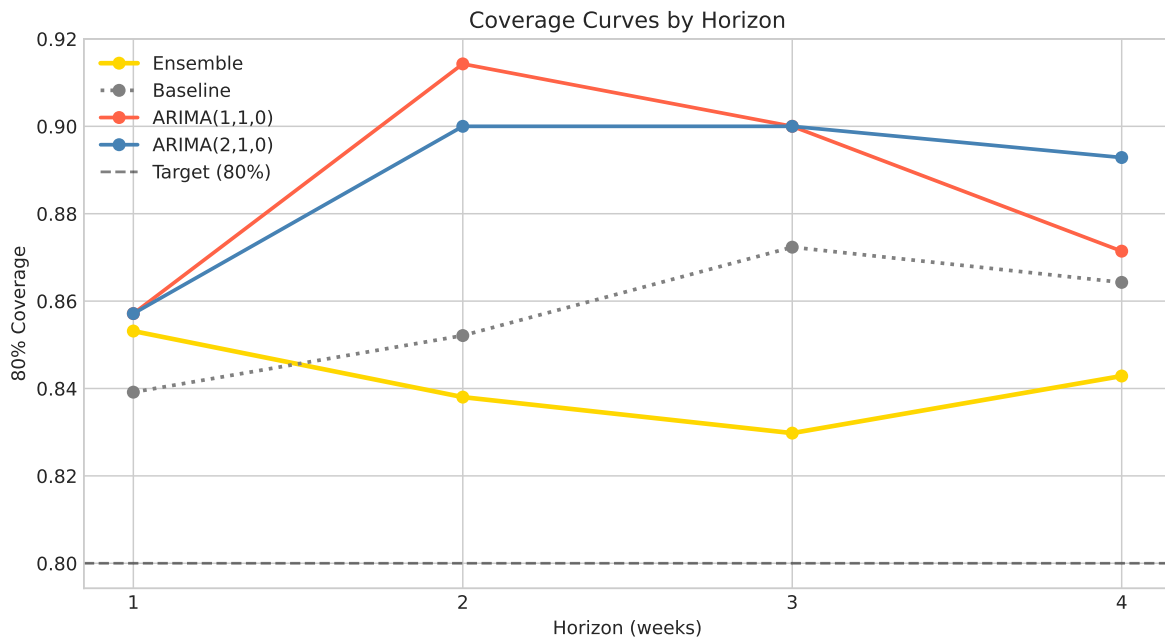

arima_coverages = {"ARIMA(1,1,0)": [], "ARIMA(2,1,0)": []}
for model_name in ["ARIMA(1,1,0)", "ARIMA(2,1,0)"]:
    for h in [1, 2, 3, 4]:
        sub = fable_fc_df[(fable_fc_df["model"] == model_name) & (fable_fc_df["h"] == h)]
        coverage = np.mean((sub["actual"] >= sub["lower"]) & (sub["actual"] <= sub["upper"]))
        arima_coverages[model_name].append(coverage)

plt.figure(figsize=(9, 5))
plt.plot(horizons, ensemble_coverages, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_coverages, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_coverages["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_coverages["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")
plt.axhline(0.80, color="black", linestyle="--", alpha=0.5, label="Target (80%)")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("80% Coverage")
plt.title("Coverage Curves by Horizon")
plt.legend()


```

```
plt.tight_layout()
plt.show()
```



(Type your discussion here)

ETS models

- (8 pts) Fit two ETS models, using `ETSModel` from the `statsmodels` package: Holt's linear trend and damped linear trend. Each of these models should have additive errors, and no seasonality. Implement time series CV to evaluate them, just as you did in Q8; you should be appending the forecast from these ETS models to `fable_fc_df`. Then, as in Q9, plot their MAE curves as a function of horizon, and overlay the curves from all models considered thus far. Discuss what you find.

```
ets_fc = []

for i in range(len(fc_dates)):
    fc_date = fc_dates[i]

    train_idx = cases_deaths_sorted[cases_deaths_sorted["date"] == fc_date].index[0]
    train_df = cases_deaths_sorted.iloc[:train_idx + 1]
    y_tr = pd.Series(train_df["deaths"].values)
```

```

future_df = cases_deaths_sorted.iloc[train_idx + 1 : train_idx + 5]
actual_deaths = future_df["deaths"].values

# 3. Holt's Linear ETS (Additive error, additive trend)
try:
    fit_holt = ETSModel(y_tr, error='add', trend='add', damped_trend=False, seasonal=None)
    pred_holt = fit_holt.get_prediction(start=len(y_tr), end=len(y_tr) + len(future_df))
    ci_holt = pred_holt.summary_frame(alpha=0.20)
    mean_holt = ci_holt["mean"].values
    lower_holt = ci_holt["pi_lower"].values
    upper_holt = ci_holt["pi_upper"].values
    for h_idx in range(len(future_df)):
        ets_fc.append({
            "model": "Holt ETS", "forecast_date": fc_date,
            "target_date": future_df.iloc[h_idx]["date"], "h": h_idx + 1,
            "mean": mean_holt[h_idx], "lower": lower_holt[h_idx],
            "upper": upper_holt[h_idx], "actual": actual_deaths[h_idx]
        })
except Exception as e:
    pass

# 4. Damped Linear ETS
try:
    fit_damped = ETSModel(y_tr, error='add', trend='add', damped_trend=True, seasonal=None)
    pred_damped = fit_damped.get_prediction(start=len(y_tr), end=len(y_tr) + len(future_df))
    ci_damped = pred_damped.summary_frame(alpha=0.20)
    mean_damped = ci_damped["mean"].values
    lower_damped = ci_damped["pi_lower"].values
    upper_damped = ci_damped["pi_upper"].values
    for h_idx in range(len(future_df)):
        ets_fc.append({
            "model": "Damped ETS", "forecast_date": fc_date,
            "target_date": future_df.iloc[h_idx]["date"], "h": h_idx + 1,
            "mean": mean_damped[h_idx], "lower": lower_damped[h_idx],
            "upper": upper_damped[h_idx], "actual": actual_deaths[h_idx]
        })
except Exception as e:
    pass

ets_fc_df = pd.DataFrame(ets_fc)
fable_fc_df = pd.concat([fable_fc_df, ets_fc_df], ignore_index=True)

```



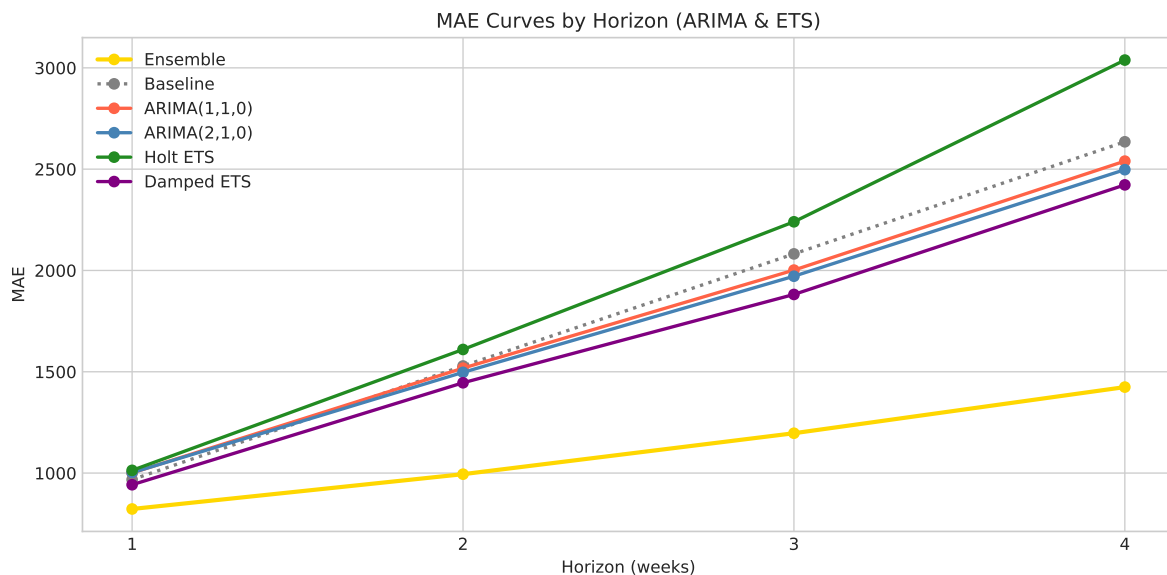
```

# Compute MAE for ETS models
ets_maes = {"Holt ETS": [], "Damped ETS": []}
for model_name in ["Holt ETS", "Damped ETS"]:
    for h in [1, 2, 3, 4]:
        sub = fable_fc_df[(fable_fc_df["model"] == model_name) & (fable_fc_df["h"] == h)]
        mae = np.mean(np.abs(sub["mean"] - sub["actual"]))
        ets_maes[model_name].append(mae)

# Plot all MAE curves
plt.figure(figsize=(10, 5))
plt.plot(horizons, ensemble_maes, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_maes, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_maes["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_maes["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")
plt.plot(horizons, ets_maes["Holt ETS"], color="forestgreen", marker="o", linewidth=2, label="Holt ETS")
plt.plot(horizons, ets_maes["Damped ETS"], color="purple", marker="o", linewidth=2, label="Damped ETS")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("MAE")
plt.title("MAE Curves by Horizon (ARIMA & ETS)")
plt.legend()
plt.tight_layout()
plt.show()

```



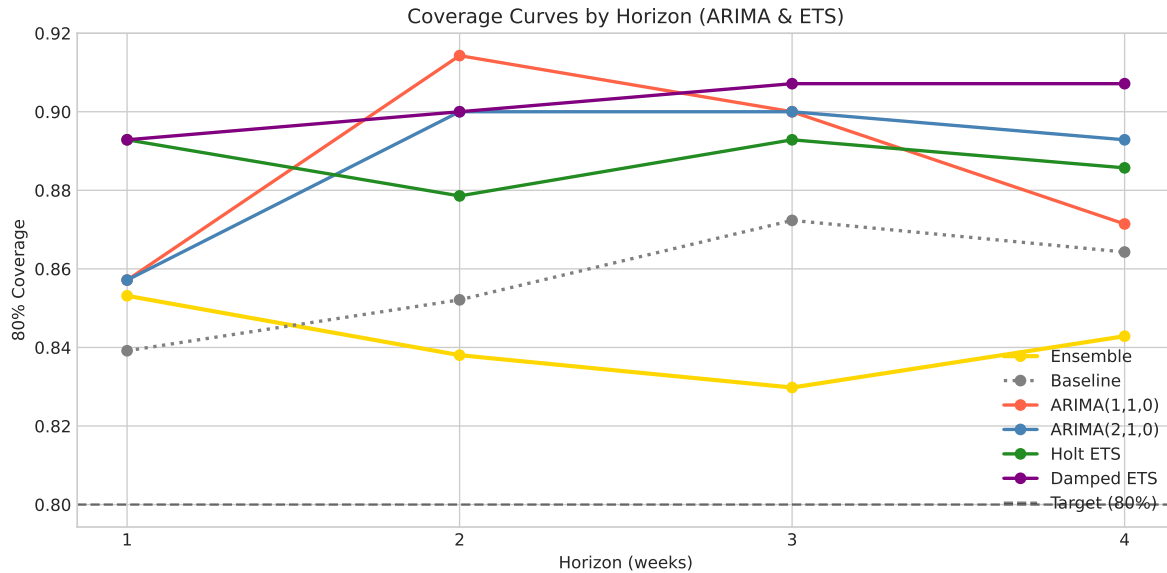
(Type your discussion here)

12. (Bonus) Compute the coverage of 80% prediction intervals from each of the ETS models, per horizon, and plot alongside the coverage curves from all models considered thus far.

```
ets_coverages = {"Holt ETS": [], "Damped ETS": []}
for model_name in ["Holt ETS", "Damped ETS"]:
    for h in [1, 2, 3, 4]:
        sub = fable_fc_df[(fable_fc_df["model"] == model_name) & (fable_fc_df["h"] == h)]
        coverage = np.mean((sub["actual"] >= sub["lower"]) & (sub["actual"] <= sub["upper"]))
        ets_coverages[model_name].append(coverage)

plt.figure(figsize=(10, 5))
plt.plot(horizons, ensemble_coverages, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_coverages, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_coverages["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_coverages["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")
plt.plot(horizons, ets_coverages["Holt ETS"], color="forestgreen", marker="o", linewidth=2, label="Holt ETS")
plt.plot(horizons, ets_coverages["Damped ETS"], color="purple", marker="o", linewidth=2, label="Damped ETS")
plt.axhline(0.80, color="black", linestyle="--", alpha=0.5, label="Target (80%)")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("80% Coverage")
plt.title("Coverage Curves by Horizon (ARIMA & ETS)")
plt.legend()
plt.tight_layout()
plt.show()
```



(Type your discussion here)

Exogenous features

13. (8 pts) As mentioned in the intro to this homework, exogenous features can play a huge role developing useful forecasts. Fortunately, here you already have a fairly obvious candidate for such an exogenous feature: reported Covid-19 case counts. Both intuitively and quantitatively (recall the cross-correlation plots that we computed near the start of the course), we know that this is a leading indicator of Covid-19 deaths. In order to be able to make up to 4 week ahead forecasts, we'll use 4-week-lagged Covid-19 cases as exogenous feature, to add to our ARIMA model. Technically, when you add an exogenous feature in a formula in the call to `ARIMA()`, this fits a regression model with ARIMA errors.

Consider a regression model of `log(deaths)` on `log(lag(cases, 4))`, with ARIMA(2,1,0) errors. The log transform is to stabilize the variance; we could have done this earlier, in our ARIMA models in the above questions, but it would have made the forecasts from our pure ARIMA models (without exogenous features) too volatile. Include an intercept (constant) term in the model. Implement time series CV for this model, using the code provided below as a scaffold. Compute the MAE per horizon, and compare this to the MAEs from the models considered thus far, on one plot. Discuss what you find—you should have taken a big leap toward the performance of the ensemble! Hint: read the comments below carefully. In order to make forecasts you will need to construct a new data set to pass to `forecast()` (as opposed to simply specifying

the horizon) because `forecast()` needs to know where to find the exogenous feature(s) to construct forecasts.

```
# Create the lagged exogenous variable
cases_deaths_sorted["x"] = cases_deaths_sorted["cases"].shift(4)
df_clean = cases_deaths_sorted.dropna(subset=["x", "deaths"]).copy().reset_index(drop=True)

exog_fc = []

for i in range(len(fc_dates)):
    fc_date = fc_dates[i]

    # Slice the train set up to the forecast date
    train_idx = df_clean[df_clean["date"] == fc_date].index[0]
    train_df = df_clean.iloc[:train_idx + 1]

    y_train_log = np.log(train_df["deaths"].values)
    X_train_log = np.log(train_df["x"].values)

    # Future values for the next 4 steps
    future_df = df_clean.iloc[train_idx + 1 : train_idx + 5]
    actual_deaths = future_df["deaths"].values
    X_future_log = np.log(future_df["x"].values)

    try:
        # Fit regression with ARIMA(2,1,0) errors
        fit_exog = ARIMA(y_train_log, exog=X_train_log, order=(2, 1, 0), trend='t').fit()

        # Forecast using future exogenous variables
        fc_res_exog = fit_exog.get_forecast(steps=len(future_df), exog=X_future_log[:len(future_df)])
        mean_exog_log = fc_res_exog.predicted_mean
        ci_exog_log = fc_res_exog.summary_frame(alpha=0.20)

        # Back-transform using median (exponential)
        mean_exog = np.exp(mean_exog_log)
        lower_exog = np.exp(ci_exog_log["mean_ci_lower"].values)
        upper_exog = np.exp(ci_exog_log["mean_ci_upper"].values)

        for h_idx in range(len(future_df)):
            exog_fc.append({
                "model": "ARIMA(2,1,0) + Cases", "forecast_date": fc_date,
                "target_date": future_df.iloc[h_idx]["date"], "h": h_idx + 1,
                "mean": mean_exog[h_idx], "lower": lower_exog[h_idx],
```

```

        "upper": upper_exog[h_idx], "actual": actual_deaths[h_idx]
    })
except Exception as e:
    pass

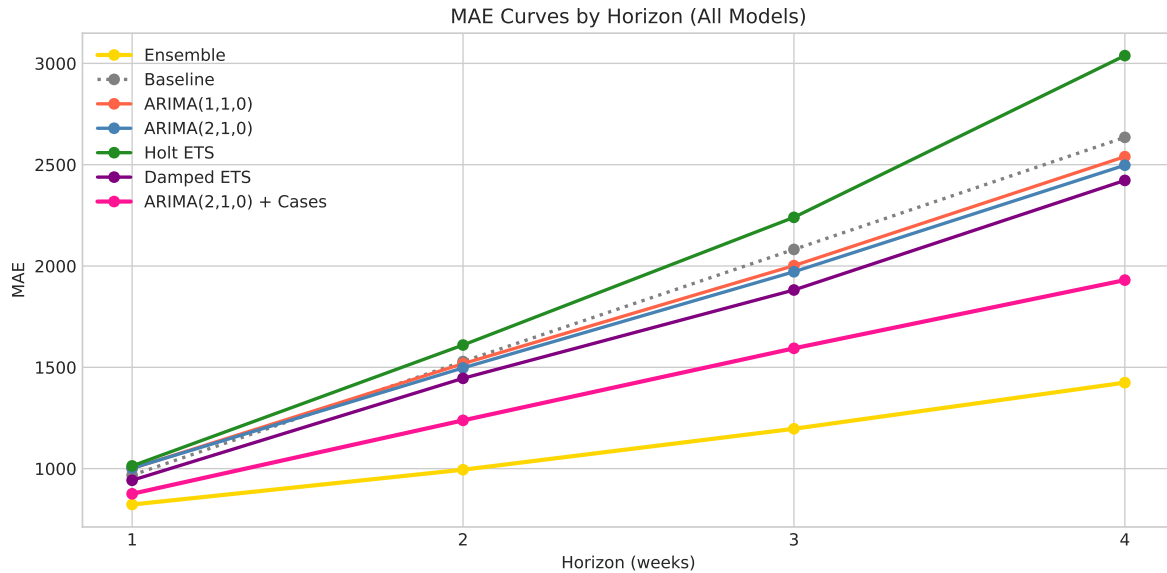
exog_fc_df = pd.DataFrame(exog_fc)
fable_fc_df = pd.concat([fable_fc_df, exog_fc_df], ignore_index=True)

# Compute MAE for the exogenous model
exog_maes = []
for h in [1, 2, 3, 4]:
    sub = fable_fc_df[(fable_fc_df["model"] == "ARIMA(2,1,0) + Cases") & (fable_fc_df["h"] == h)]
    mae = np.mean(np.abs(sub["mean"] - sub["actual"]))
    exog_maes.append(mae)

# Plot all MAE curves including the exogenous model
plt.figure(figsize=(10, 5))
plt.plot(horizons, ensemble_maes, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_maes, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_maes["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_maes["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")
plt.plot(horizons, ets_maes["Holt ETS"], color="forestgreen", marker="o", linewidth=2, label="Holt ETS")
plt.plot(horizons, ets_maes["Damped ETS"], color="purple", marker="o", linewidth=2, label="Damped ETS")
plt.plot(horizons, exog_maes, color="deeppink", marker="o", linewidth=2.5, label="ARIMA(2,1,0) + Cases")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("MAE")
plt.title("MAE Curves by Horizon (All Models)")
plt.legend()
plt.tight_layout()
plt.show()

```



(Type your discussion here)

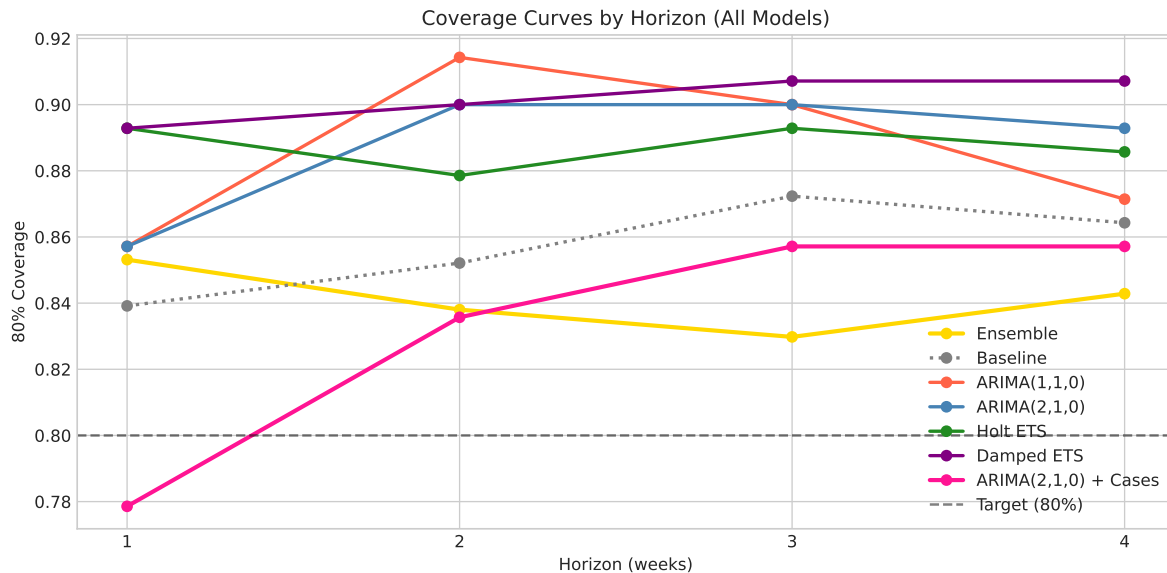
14. (Bonus) Compute the coverage of 80% prediction intervals from the ARIMA model with the exogenous feature (cases), and plot alongside the coverage curves from models considered thus far.

```
exog_coverages = []
for h in [1, 2, 3, 4]:
    sub = fable_fc_df[(fable_fc_df["model"] == "ARIMA(2,1,0) + Cases") & (fable_fc_df["h"] == h)]
    coverage = np.mean((sub["actual"] >= sub["lower"]) & (sub["actual"] <= sub["upper"]))
    exog_coverages.append(coverage)

plt.figure(figsize=(10, 5))
plt.plot(horizons, ensemble_coverages, color="gold", marker="o", linewidth=2.5, label="Ensemble")
plt.plot(horizons, baseline_coverages, color="grey", marker="o", linestyle=":", linewidth=2, label="Baseline")
plt.plot(horizons, arima_coverages["ARIMA(1,1,0)"], color="tomato", marker="o", linewidth=2, label="ARIMA(1,1,0)")
plt.plot(horizons, arima_coverages["ARIMA(2,1,0)"], color="steelblue", marker="o", linewidth=2, label="ARIMA(2,1,0)")
plt.plot(horizons, ets_coverages["Holt ETS"], color="forestgreen", marker="o", linewidth=2, label="Holt ETS")
plt.plot(horizons, ets_coverages["Damped ETS"], color="purple", marker="o", linewidth=2, label="Damped ETS")
plt.plot(horizons, exog_coverages, color="deeppink", marker="o", linewidth=2.5, label="ARIMA(2,1,0) + Cases")
plt.axhline(0.80, color="black", linestyle="--", alpha=0.5, label="Target (80%)")

plt.xticks(horizons)
plt.xlabel("Horizon (weeks)")
plt.ylabel("80% Coverage")
plt.title("Coverage Curves by Horizon (All Models)")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



(Type your discussion here)

15. (Bonus) For the regression model from Q13 (pick a single regression model that was fit at a single iteration of time series CV), check that its 1 week ahead point forecast matches the formulas you know underlie it. As before, you'll have to extract the coefficients from the fitted model, and then form the 1 week ahead predictions manually, verifying that you get the same result. Hint: here you should take the forecast to be the median of the forecast distribution and not the mean. Because of the data transformation (we're modeling log deaths), the `fable` package will do something to adjust the mean after back-transforming which is nonobvious. [This](#) page gives more details.

```
# Pick the model fit at the first iteration of CV (June 6, 2020)
train_idx = df_clean[df_clean["date"] == "2020-06-06"].index[0]
train_df = df_clean.iloc[:train_idx + 1]

y_train_log = np.log(train_df["deaths"].values)
X_train_log = np.log(train_df["x"].values)

# Exogenous case value for next step (index target date June 13, 2020)
next_row = df_clean.iloc[train_idx + 1]
X_next_log = np.log(next_row["x"])
```

```

# Fit ARIMA(2,1,0) on logs with trend='t'
fit = ARIMA(y_train_log, exog=X_train_log, order=(2, 1, 0), trend='t').fit()
fc = fit.forecast(steps=1, exog=[X_next_log])

y_t = y_train_log[-1]
y_t_1 = y_train_log[-2]
y_t_2 = y_train_log[-3]
X_t = X_train_log[-1]
X_t_1 = X_train_log[-2]
X_t_2 = X_train_log[-3]

# Coefficients
c = fit.params[0]
beta = fit.params[1]
phi1 = fit.params[2]
phi2 = fit.params[3]

# Residuals of the differenced equation
# Y_t = y_t - y_{t-1}
# DX_t = X_t - X_{t-1}
# Z_t = Y_t - beta * DX_t - c
Y_t = y_t - y_t_1
DX_t = X_t - X_t_1
Z_t = Y_t - beta * DX_t - c

Y_t_1 = y_t_1 - y_t_2
DX_t_1 = X_t_1 - X_t_2
Z_t_1 = Y_t_1 - beta * DX_t_1 - c

# Manual forecast calculation
Z_next_pred = phi1 * Z_t + phi2 * Z_t_1
DX_next = X_next_log - X_t
Y_next_pred = c + beta * DX_next + Z_next_pred
manual_fc_log = y_t + Y_next_pred

# Back-transformed median forecasts (using exponential)
fc_median = np.exp(fc[0])
manual_fc_median = np.exp(manual_fc_log)

print(f"Regression + ARIMA errors 1-week ahead forecast (Log scale):")
print(f"  Statsmodels = {fc[0]:.6f}")
print(f"  Manual = {manual_fc_log:.6f}")

```



```
print(f"\nBack-transformed Median Forecasts (Original scale):")
print(f"  Statsmodels = {fc_median:.4f}")
print(f"  Manual = {manual_fc_median:.4f}")
```

Regression + ARIMA errors 1-week ahead forecast (Log scale):
 Statsmodels = 8.506858
 Manual = 8.506858

Back-transformed Median Forecasts (Original scale):
 Statsmodels = 4948.5883
 Manual = 4948.5883

(Type your discussion here)

16. (Bonus) Repurposing the code used to plot the ensemble forecasts given at the start of the homework, plot forecasts from the 5 **fable** models that you've developed, at the same set of forecast dates. (You should have 5 separate plots.)

```
models = ["ARIMA(1,1,0)", "ARIMA(2,1,0)", "Holt ETS", "Damped ETS", "ARIMA(2,1,0) + Cases"]

for model_name in models:
    plot_forecast_trajectories(
        cases_deaths_sorted,
        fable_fc_df[fable_fc_df["model"] == model_name],
        f"Forecasts trajectories for {model_name}",
        fc_dates=fc_dates[:,6]
    )
```

